

1. Write a C program to implement daytime client/server program using TCP sockets
Server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <netinet/in.h>
#define PORT 9090 // You can use port 13 for RFC compliance, but 9090 is safer for
testing

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[128];

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == 0) {
        perror("Socket failed");
        exit(EXIT_FAILURE);
    }

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);

    printf("Daytime server listening on port %d...\n", PORT);

    while (1) {
        new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t
*)&addrlen);
        if (new_socket < 0) {
            perror("Accept failed");
            continue;
        }

        time_t now = time(NULL);
        struct tm *tm_info = localtime(&now);
        strftime(buffer, sizeof(buffer), "%Y-%m-%d %H:%M:%S\n", tm_info);

        write(new_socket, buffer, strlen(buffer));
    }
}
```

```

        close(new_socket);
    }

    close(server_fd);
    return 0;
}

```

Client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

```

```

#define PORT 9090

```

```

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[128] = {0};

    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        perror("Socket creation error");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    // Connect to localhost
    if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
        perror("Invalid address / Address not supported");
        return -1;
    }

    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Connection failed");
        return -1;
    }

    read(sock, buffer, sizeof(buffer));
    printf("Server time: %s", buffer);

    close(sock);
    return 0;
}

```

```
}
```

How to Compile and Run:

Compile: gcc daytime_server.c -o daytime_server

gcc daytime_client.c -o daytime_client

Run:

1. Start the **server** in one terminal: ./daytime_server

2. Run the **client** in another terminal: Run the **client** in another terminal:

2. Write a TCP client/server program in which client sends three numbers to the server in a single message. Server returns sum, difference and product as a result single message. Client program should print the results appropriately

Server.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <netinet/in.h>
```

```
#define PORT 8080
```

```
int main() {
```

```
    int server_fd, new_socket;
```

```
    struct sockaddr_in address;
```

```
    int addrlen = sizeof(address);
```

```
    int buffer[3]; // To receive 3 integers
```

```
    int result[3]; // sum, difference, product
```

```
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

```
    if (server_fd == 0) {
```

```
        perror("Socket failed");
```

```
        exit(EXIT_FAILURE);
```

```
    }
```

```
    address.sin_family = AF_INET;
```

```
    address.sin_addr.s_addr = INADDR_ANY;
```

```
    address.sin_port = htons(PORT);
```

```
    bind(server_fd, (struct sockaddr*)&address, sizeof(address));
```

```
    listen(server_fd, 3);
```

```
    printf("Server listening on port %d...\n", PORT);
```

```
    new_socket = accept(server_fd, (struct sockaddr*)&address, (socklen_t*)&addrlen);
```

```
    read(new_socket, buffer, sizeof(buffer));
```

```

int a = buffer[0], b = buffer[1], c = buffer[2];
result[0] = a + b + c;
result[1] = a - b - c;
result[2] = a * b * c;

write(new_socket, result, sizeof(result));
printf("Sent results to client.\n");

close(new_socket);
close(server_fd);
return 0;
}

```

Client.C

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080

int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    int buffer[3] = {5, 3, 2}; // You can change these numbers
    int result[3]; // To store sum, difference, product

    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) {
        printf("\n Socket creation error \n");
        return -1;
    }

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);

    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    if (connect(sock, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
        printf("\nConnection Failed \n");
        return -1;
    }
}

```

```

write(sock, buffer, sizeof(buffer));
read(sock, result, sizeof(result));

printf("Sent numbers: %d, %d, %d\n", buffer[0], buffer[1], buffer[2]);
printf("Sum: %d\n", result[0]);
printf("Difference: %d\n", result[1]);
printf("Product: %d\n", result[2]);

close(sock);
return 0;
}

```

How to run : compile both

`gcc server.c -o server`

`gcc client.c -o client`

Start server

`./server`

Run Client

`./client`

3. Write a C program that prints the IP layer and TCP layer socket options in a separate file

`print_socket_options.c`

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <netinet/in.h>
#include <netinet/tcp.h>
#include <sys/socket.h>
#include <errno.h>

void print_error(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

int main() {
    int sockfd;
    FILE *fp = fopen("socket_options.txt", "w");
    if (!fp) {
        perror("File open error");
        exit(EXIT_FAILURE);
    }
}

```

```

sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0) {
    print_error("Socket creation failed");
}

// IP layer options
int ip_ttl;
socklen_t optlen = sizeof(ip_ttl);
if (getsockopt(sockfd, IPPROTO_IP, IP_TTL, &ip_ttl, &optlen) < 0)
    print_error("getsockopt IP_TTL");
fprintf(fp, "IP_TTL: %d\n", ip_ttl);

int ip_hdrincl;
optlen = sizeof(ip_hdrincl);
if (getsockopt(sockfd, IPPROTO_IP, IP_HDRINCL, &ip_hdrincl, &optlen) < 0)
    fprintf(fp, "IP_HDRINCL: Not applicable to TCP (expected error: %s)\n",
strerror(errno));
else
    fprintf(fp, "IP_HDRINCL: %d\n", ip_hdrincl);

// TCP layer options
int tcp_nodelay;
optlen = sizeof(tcp_nodelay);
if (getsockopt(sockfd, IPPROTO_TCP, TCP_NODELAY, &tcp_nodelay, &optlen) <
0)
    print_error("getsockopt TCP_NODELAY");
fprintf(fp, "TCP_NODELAY: %d\n", tcp_nodelay);

int tcp_maxseg;
optlen = sizeof(tcp_maxseg);
if (getsockopt(sockfd, IPPROTO_TCP, TCP_MAXSEG, &tcp_maxseg, &optlen) < 0)
    print_error("getsockopt TCP_MAXSEG");
fprintf(fp, "TCP_MAXSEG: %d\n", tcp_maxseg);

#ifdef TCP_CORK
int tcp_cork;
optlen = sizeof(tcp_cork);
if (getsockopt(sockfd, IPPROTO_TCP, TCP_CORK, &tcp_cork, &optlen) == 0)
    fprintf(fp, "TCP_CORK: %d\n", tcp_cork);
#endif

#ifdef TCP_KEEPIDLE
int keep_idle;
optlen = sizeof(keep_idle);
if (getsockopt(sockfd, IPPROTO_TCP, TCP_KEEPIDLE, &keep_idle, &optlen) == 0)
    fprintf(fp, "TCP_KEEPIDLE: %d seconds\n", keep_idle);

```

```

#endif

#ifdef TCP_KEEPINTVL
    int keep_intvl;
    optlen = sizeof(keep_intvl);
    if (getsockopt(sockfd, IPPROTO_TCP, TCP_KEEPINTVL, &keep_intvl, &optlen) ==
0)
        fprintf(fp, "TCP_KEEPINTVL: %d seconds\n", keep_intvl);
#endif

#ifdef TCP_KEEPCNT
    int keep_cnt;
    optlen = sizeof(keep_cnt);
    if (getsockopt(sockfd, IPPROTO_TCP, TCP_KEEPCNT, &keep_cnt, &optlen) == 0)
        fprintf(fp, "TCP_KEEPCNT: %d probes\n", keep_cnt);
#endif

    fclose(fp);
    close(sockfd);

    printf("Socket options written to socket_options.txt\n");
    return 0;
}

```

How to Compile and Run

```
gcc print_socket_options.c -o print_socket_options
```

```
./print_socket_options
```

Then check the output file:

```
cat socket_options.txt
```

Notes:

- IP_TTL is the default Time To Live for packets sent from the socket.
- TCP_NODELAY controls Nagle's algorithm.
- TCP_MAXSEG is the maximum segment size.
- Some options (like TCP_CORK, TCP_KEEPIIDLE) may not be available on all systems — those are included conditionally.